SCALING EMPOWERED AGENTS

Anonymous authors
Paper under double-blind review

ABSTRACT

Empowerment measures the capacity of an agent to actively control its environment. In this work, we explain the connection between skill learning and empowerment. While skill learning is not empowerment, we introduce a simple scalable recipe for empowerment maximization: empowerment corresponds to the number of executable skills from a state, so maximizing empowerment corresponds to reaching states that are good starting points for many skills. Our recipe applies to arbitrary underlying skill learning methods. Whereas most prior work on empowerment looks at tabular settings, this recipe enables us to study empowerment at a scale where we can realize theoretically-predicted but empirically-elusive properties of empowerment (e.g., information gathering, survival, and homeostasis). Experiments show that empowerment maximization unlocks interesting emergent properties, such as building a house in Crafter. Code and videos at `tinyurl.com/2a7nzbrb`.

1 INTRODUCTION

Empowerment, a measure of agency, has been speculated as a unified mathematical explanation for myriad properties of intelligent agents, ranging from building causal models (Gopnik, 2024; Abel et al., 2026; Lidayan et al., 2025), acquiring resources (Mohamed and Jimenez Rezende, 2015), maintaining homeostasis (Mohamed and Jimenez Rezende, 2015), power-seeking (Turner et al., 2019), to explanations of how children explore (Lidayan et al., 2025). Prior work argues that empowerment should be especially important in open-ended environments, but does not demonstrate empowerment maximization in such settings. However, despite decades of research studying the behavior of empowerment-maximizing agents, there has not yet been empirical demonstrations of algorithmic empowerment maximization in the "big world" (Javed and Sutton, 2024) – challenging, large settings where behaviors like survival and resource collection become tough. It is only in such rich tasks where we expect to see emergent behaviors from empowerment maximization.

Agents should successfully interact with an environment that they cannot fully describe. For example, even if a young child cannot fully explain all of the intricacies of a game like Minecraft, the child will still become empowered by defeating enemies and building a house, reaching an empowered state from which they know how to do many things. However, maximizing empowerment is a tricky computational problem. First, empowerment maximization requires solving for empowerment at particular contexts. If the context is a state, this means solving for empowerment in all of reachable state space. Second, solving for the empowerment at a particular context is a challenging problem and requires not only modeling the relationship between actions and outputs (e.g. a transition model), but also finding the actions that maximize control. While tractable using standard algorithms in tabular settings, the combination of these difficulties means canonical methods like Blahut-Arimoto cannot exactly solve this computational problem in high-dimensional settings. In the last decade, methods like VIM (Mohamed and Jimenez Rezende, 2015), variational empowerment (Karl et al., 2017), and LatentGCE (Zhao et al., 2021) aimed to address this computational problem using deep learning, and were the first to demonstrate learning-based empowerment maximization in simple tabular and continuous settings. However, there has not yet been a stable empowerment-maximization method that builds on the foundation of these works and scales to large continuous or discrete settings. This gap is important to resolve, given that large settings are precisely the right environments for empowerment maximization – where an agent must execute intelligent behavior despite having a limited view of a complex world.

In this work, we scale empowerment maximization to a regime where theorized intelligent behaviors can emerge. Our starting point is the observation that empowerment maximization is closely related

to unsupervised skill discovery (USD) methods. The goal of empowerment maximization is to reach states from which it is easy to reach a diverse set of states. Relatedly, USD methods aim to learn a diverse set of different "skills". However, unlike prior empowerment-maximization methods that have struggled to scale, modern skill discovery methods can learn interpretable, diverse behaviors in continuous, high-dimensional tasks. As we will show, a particular class of USD methods called mutual information skill learning (MISL) measures the point-wise empowerment: at any particular state, the number of different skills an agent can execute from that state lower bounds its empowerment. In other words, an agent is empowered at states where it has many options, or skills. Thus, while skill learning methods do not directly learn policies that maximize empowerment, we can use scalable skill learning methods to estimate the empowerment of a particular state; we can then train another policy that ascends the empowerment landscape to get to states where it has many options.

Our work has two key contributions. **First**, we provide a general recipe to convert scalable skill-learning methods into empowerment maximization methods and show our methods scale when baselines cannot. **Second**, we demonstrate our method in challenging, high-dimensional settings where empowerment-maximizing behavior becomes nontrivial, and analyze the behavior of empowered agents including resource gathering, survival, and environment modification. For example, early in training in Crafter, empowerment-maximizing agents learn to build houses at night when enemies appear and make and use tools. **Our work shows that scalable empowerment maximization is possible, demonstrating interesting emergent behaviors in complex, open-ended setting.**

2 Related Work

Intrinsic Motivation Exploration Methods. Intrinsic motivation enables exploration without external information about the Markov decision process (MDP) (Gruaz et al., 2024). Prior intrinsic motivation methods reward curiosity (Pathak et al., 2017), novelty (Bellemare et al., 2016), information gain (Houthooft et al., 2016), and prediction error (Burda et al., 2019). Empowerment instead rewards agency and control (Mohamed and Jimenez Rezende, 2015), a motivation thought to explain human behavior and desired in complex, open-ended settings (Gopnik, 2024; Yiu et al., 2026; Dai and Gopnik, 2025; Lidayan et al., 2025; Yang et al., 2026). Prior works predicts that empowerment maximization should generate behaviors such as (1) environment modification (Salge et al., 2014a), (2) resource gathering (Mohamed and Jimenez Rezende, 2015), and (3) survival. While these emergent behaviors have been previously demonstrated in simple experiments, we present scalable empirical evidence in large, open-ended settings.

Potential and Effective Empowerment. As originally introduced, empowerment quantifies the influence an agent's open-loop actions can exert on future states (Klyubin et al., 2005b; Salge et al., 2014b). Two complementary forms of empowerment are useful (Capdepuy, 2011; Ji et al., 2026). *Potential empowerment* measures the channel capacity between a sequence of actions and future observations conditioned on a starting state, measuring the *maximal possible* control from a particular context. In contrast, *effective empowerment* measures how many bits a particular distribution over open-loop controls communicates to the future. Potential empowerment is a scalar property of the context, while effective empowerment is also a function of the controls.

Prior Deep Learning Methods for Estimating and Maximizing Potential Empowerment. VIM maximizes a variational lower bound on open-loop empowerment by learning a decoder that infers action sequences from policy outcomes (Mohamed and Jimenez Rezende, 2015). Later empowerment maximization works like LatentGCE alternatively learn latent forward models to estimate the same empowerment, but relies on sufficient coverage over actions sequences in high-empowerment regions (Zhao et al., 2021). This requirement becomes increasingly demanding in higher-dimensional tasks. In contrast, we study *closed-loop* empowerment (Gregor et al., 2017; Tiomkin et al., 2025), which is the channel capacity of an information channel from a collection of closed-loop policies to outcomes. Thus, a state is empowering if an agent can execute a diverse repertoire of reactive policies from that state. This closed-loop formulation means empowerment measures what a standard reinforcement learning (RL) agent can actually do and naturally connects with unsupervised skill-discovery methods.

Unsupervised Skill Discovery. MISL methods aim to learn diverse policies that maximize a mutual information (MI) between skill latents $z \in \mathcal{Z}$ and the outcomes of latent-conditioned policies

(i.e. $\pi(a \mid s, z)$). Modern MISL methods learn policies that maximize MI objectives over one-step transitions $I(Z; S, S')$ ((Zheng et al., 2025; Park et al., 2023; 2026)), long-horizon transitions $I(Z; S_\tau \mid S_0)$ for termination time τ (Gregor et al., 2017), occupancies $I(Z; S)$, (Eysenbach et al., 2018) and the conditional MI $I(Z; S' \mid S)$ (Sharma et al., 2020). Typically, these methods learn a classifier that identifies skills from outcomes and use this classifier as a reward, so that the expected value function is a variational lower bound on the MI. These methods scale to high-dimensional locomotion and pixel-based tasks.

Skill Learning and Effective Empowerment. The MI maximized in MISL methods are related to closed-loop effective empowerment. Skills are *not* potential empowerment-maximizing policies, but they can provide estimates of the potential empowerment. Despite this relationship and the scalability of modern skill learning methods, recent skill-learning papers do not reference empowerment (Zheng et al., 2025; Park et al., 2023; 2026). Furthermore, while recent works have called attention to this relationship between skill-learning and empowerment (Levy et al., 2025), to our knowledge, there has not yet been work that exploits this connection to do scalable potential empowerment maximization. We close this gap by adapting modern MISL methods to maximize potential empowerment in large, open-ended settings.

3 Defining Empowerment

Our work focuses on a controlled Markov process (i.e., an MDP without a reward function) with states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, and environment dynamics $p(s' \mid s, a)$. Our goal is to learn a *potential empowerment* policy π_{pot} that visits states with high potential empowerment $\mathcal{E}_{\text{pot}}(s)$:

$$\pi_{\text{pot}} \in \arg\max_{\pi \in \Pi} \mathbb{E}_\pi \left[\sum_t \gamma^t \mathcal{E}_{\text{pot}}(s_t) \right]. \tag{1}$$

Empowerment $\mathcal{E}_{\text{pot}}(s_0)$ is typically (Salge et al., 2014b) defined as the maximal conditional MI between behavior Z and future states S_+, given an initial state s_0: $\mathcal{E}_{\text{pot}}(s_0) = \max_{p(A)} I(A; S_+ \mid S_0 = s_0)$. However, prior work differs on what exactly constitutes the behavior Z: is it the immediate next action (Salge et al., 2014b, Sec. 4.4.2), a sequence of actions (Klyubin et al., 2005b), or the index of a closed-loop policy (Gregor et al., 2017; Tiomkin et al., 2025)? Prior work also differs in how precisely the future state S_+ is sampled. This paper will adopt an expressive definition of empowerment that measures how behaviors described by closed-loop policies control states *many* steps into the future.

To make the definition of behavior precise, we will consider a z and context-conditioned policy $\pi(a \mid s, z; s_0) \in \Pi_{\mathcal{Z}}$, with s_0 taken to be the starting context (in our case, a starting state s_0) of the behavior. For a specific "skill vector" z we will refer to this policy as a "skill". Importantly, the context state s_0 and the current state s are two different objects – the context determines the "start point" of the skill, while current state tells the agent what to do now.

Let $p(z)$ be a distribution over these skills, which we will assume to be uniform (without loss of generality (Ji et al., 2026)). By introducing $p(z)$ as a distribution over skill vectors, we define a distribution over the space of policies. The term "skill" comes from the fact that this distribution over the space of policies may assign zero probability to almost all policies, so the skills are the set of policies with non-zero probability (Eysenbach et al., 2021). For each of these skills $\pi(a \mid s, z)$, we define the discounted state occupancy measure (DSOM) as:

$$p^\pi(s_+ = s \mid s_0, z) \triangleq (1-\gamma) \sum_{t=0}^{\infty} \gamma^t p^\pi(s_{t+1} = s \mid s_0, z). \tag{2}$$

Averaging over all skills gives

$$p^\pi(s_+ \mid s_0) = \mathbb{E}_{p(z)} \left[p^\pi(s_+ \mid s_0, z) \right]. \tag{3}$$

We now define the *potential* empowerment using the DSOM definitions:

Definition 1 (Potential empowerment)**.** *Let $p(z)$ be given, and let $S_+ \sim p^\pi(s_+ \mid s_0, z)$ be a sample from the DSOM of skill $\pi(a \mid s, z)$. Then, the potential empowerment of state s is the conditional MI*

between the skill vector z and the future state S_+:

$$\mathcal{E}_{\text{pot}}(s_0) \triangleq \max_{\pi \in \Pi_{\mathcal{Z}}} \underbrace{I^\pi(Z; S_+ \mid S_0 = s_0)}_{\triangleq \mathcal{E}_{\text{eff}}(\pi, s_0)}. \tag{4}$$

The potential empowerment can be understood as the maximum number of bits that the skill vector Z can broadcast into the future (S_+), starting from state s_0. Our goal is to learn a policy π_{pot} that uses this empowerment as a reward, effectively navigating to states where it can broadcast many bits into the future (i.e., exert a strong influence on future outcomes).

Estimating potential empowerment requires solving the inner maximization problem over $\Pi_{\mathcal{Z}}$. We will refer to the objective for this inner optimization problem as *effective empowerment*, $\mathcal{E}_{\text{eff}}(\pi, s_0)$. **We observe that this inner optimization problem is similar to that solved in prior works on MISL** (Gregor et al., 2017; Zheng et al., 2025). Whereas potential empowerment measures how good a *state* is, effective empowerment measures how good a *set of policies* is from a particular state. Of course, if this inner optimization problem is only solved approximately, then we obtain a lower bound on the empowerment.

4 Skills Estimate Empowerment (SEE): A Scalable Algorithm for Maximizing Empowerment

In this section, we introduce a scalable, online algorithm (SEE) to maximize empowerment (Eq. 1) by exploiting the connection between skill learning and empowerment. This connection between learning skills (Sec. 4.1) and maximizing potential empowerment (Sec. 3) suggests a two-policy algorithm. The skill policy π_{eff} estimates a lower bound $\widehat{\mathcal{E}_{\text{pot}}}(s)$ on the state-wise potential empowerment, while the potential policy π_{pot} maximizes this estimate as an intrinsic reward. The effective and potential empowerment policies support one another: the skills π_{eff} measure the controllable futures available from a state, while the potential empowerment policy π_{pot} moves the agent toward states from which the agent can exert control.

4.1 The Inner Optimization: Learning Skills

We modify prior MISL methods (Zheng et al., 2025) to maximize effective empowerment, which lower bounds potential empowerment by Definition 1. Recall that the expectation of a classifier that identifies skills from transitions over the course of the trajectory is a variational lower bound on the MI. This mathematical observation implies that setting the reward to the skill classifier gives a value function that, in expectation, is a lower bound on the MI. SEE builds on contrastive successor features (CSF) (Zheng et al., 2025), a recent state-of-the-art skill-learning method, to approximately lower bound the effective empowerment $\mathcal{E}_{\text{eff}}(\pi, s_0) = I^\pi(Z; S_+ \mid S_0 = s_0)$ for a fixed starting state s_0. CSF estimates the one-step MI $I(Z; S, S')$ from a classifier that identifies continuous skills from one-step transitions $s \to s'$. To capture long-horizon dependence between skills and future states, we instead train the classifier on transitions separated by a geometric random variable lengths of time $\tau \sim \text{Geom}(1-\gamma)$:

$$C_\phi(z; s_0, s_+) \approx \log \frac{q_\phi(z \mid s_0, s_+)}{p(z)}. \tag{5}$$

Following CSF (Zheng et al., 2025), Metric-aware Abstraction (METRA) (Park et al., 2023), and GenDa (Park et al., 2026), SEE parameterizes the classifier logits with a bilinear form of the learned representations ϕ. This parameterization means that skills are only distinguishable if learned representations linearly separate over the course of skill execution. Prior work find that this parameterization is important to generate sustained, space-filling behaviors (Park et al., 2023). SEE utilizes InfoNCE (Oord et al., 2018) to estimate the classifier. Additional discussion on this parameterization and its implications are in Section A.1.

Accumulating this classifier over the trajectory gives, in expectation, a variational lower bound on the MI. For a skill and start-state conditioned policy $\pi(a \mid s, z; s_0)$, define the effective empowerment Q-

and value function as

$$Q_{\text{eff}}(s, a; s_0, z) \triangleq (1-\gamma)\, \mathbb{E}_{\pi(a|s,z;s_0)} \left[\sum_{k=0}^{\infty} \gamma^k \log \frac{q(z \mid s_0, s_{t+k+1})}{p(z)} \middle| s_t = s,\ a_t = a \right] \tag{6}$$

$$V_{\text{eff}}(s; s_0, z) \triangleq \mathbb{E}_{\pi(a|s,z;s_0)} \left[Q_{\text{eff}}(s, a; s_0, z) \right]. \tag{7}$$

Then, in expectation over a uniform skill source $p(z)$ initialized at the start-state s_0, we have that

$$\mathbb{E}_{p(z)}[V_{\text{eff}}(s_0; s_0, z)] \leq \mathcal{E}_{\text{eff}}(\pi, s_0), \tag{8}$$

so the expected value function lower bounds the effective empowerment. Potential empowerment then uses a Monte-Carlo estimate of $\mathbb{E}_{p(z)}[V_{\text{eff}}(s_0; s_0, z)] \approx \frac{1}{k} \sum_{z_1 \cdots z_k \sim p(z)} V_{\text{eff}}(s_0; s_0, z_k)$ to get an estimate of the pointwise potential empowerment:

$$\hat{\mathcal{E}_{\text{eff}}}(\pi, s_0) = \mathbb{E}_{p(z)}[V_{\text{eff}}(s_0; s_0, z)] \leq I(Z; S_+ \mid S_0 = s_0) \leq \mathcal{E}_{\text{pot}}(s_0). \tag{9}$$

4.2 Practical Considerations

Conditional and Joint MI Classifiers. While empowerment is theoretically a conditional MI, in practice, we found that training policies with classifiers that identify skills from both s_0 and s_+ led to stronger performance in continuous benchmarks. We include implementations of SEE using both classifiers, using the joint classifier by default in continuous settings and the conditional-classifier by default in Crafter (Section A.2).

Limiting empowerment context. As noted in (Klyubin et al., 2005b; Abel et al., 2026), empowerment depends on an agent's particular perception. Otherwise, an agent may obtain high empowerment by perturbing controllable but meaningless variables. We therefore run experiments that restrict variables over which empowerment is defined. This restriction determines which changes in the state are salient to the skill, without restricting the information available to the policy for control. Thus, the classifier may receive different degrees of freedom from the full-observation policy and critic.

Two-stage rollout. For large state spaces, we cannot simply learn skills for every starting state s_0 and then maximize the potential empowerment over the constructed field. Instead, we consider a two-stage rollout scheme: within each episode of data collection, we first roll out the potential empowerment policy and then roll out one of the effective empowerment policies (i.e., sample a skill z and execute $\pi_{\text{eff}}(a \mid s, z; s_0)$) (Sections A.2 and A.4). We can interpret skills as local probes of empowerment, where the potential empowerment policy ascends the empowerment landscape estimated by these probes.

Method Summary. We adapt CSF, a scalable MISL method, to learn skills, which gives us (a lower bound on) the potential empowerment of states $\mathcal{E}(s)$. Then, we train a second potential empowement policy using the estimated potential empowerment as a reward. This policy navigates to states *from which* it can exert control.

5 Experiments

We (1) show scalability of the method, (2) carefully ablate and analyze all of the salient design decisions (and provide open-source implementations for future development), and (3) interpret the resulting emergent behaviors. All training setups and hyperparameters are in Sections B.1 and B.2.

Environments. To test whether the method works in large, open-ended settings, we implement our method in Craftax-Classic (Matthews et al., 2024) (JAX-accelerated Crafter). Crafter is a large and partially observed setting with a 1345-dimensional symbolic observation and 17 discrete actions (Fig. 2d) consisting of crafting, resource collection, fighting mobs, and eating/drinking. In this environment, we define the empowerment degrees of freedom to be over the agent inventory. The reason for this choice is that we do *not* wish for an agent to be empowered over features like e.g. health, hunger, and thirst, which would imply having a lot of control over these precise status values (for instance, a skill that *decreases* health).

To test whether the method works in continuous settings, we chose two high-DoF benchmarks that capture locomotion and manipulation. The AntSoccer environment consists of an ant and a ball in a closed arena, where an ant can freely kick the ball (Park et al., 2025). Within the AntSoccer environment, we consider the setting where empowerment is defined over all 42 degrees-of-freedom (e.g. joint angles, xy position, ball position, "AntSoccer") and strictly the ball position ("Ant"). BuilderBench (Ghugare et al., 2026) consists of a gripper with cubes. We consider an environment with 3 cubes in the corner of a bounded play area (Fig. 2a, white corners denote the boxed play area), where empowerment is defined over the 9 degrees-of-freedom that denote the cube positions.

Baselines. We consider two baselines that learn potential empowerment policies: LatentGCE (Zhao et al., 2021) and VIM (Mohamed and Jimenez Rezende, 2015). Both of these methods learn policies that navigate to high open-loop empowerment regions (Section C). We omit standard MISL works because they optimize effective empowerment policies from a fixed initial state distribution and do not learn a potential empowerment policy, though these methods bear similarity to our skill-switching ablation.

Ablations. We ablate key design decisions in SEE. While our method builds on MISL, there are fundamental differences between empowerment maximization and skill learning. Skill learning trains effective empowerment policies *usually from a fixed or slightly noised starting state*. Potential empowerment maximization is *not* as simple as running MISL out-of-the-box and deals with the tougher problem of learning skills originating over many regions of state space. Design decisions such as skill-switching and the choice of the underlying MISL parameterization are critical to learn behaviorally interesting policies. We do the following ablations:

- **skill-switching** for learning the potential empowerment policy,
- **joint versus conditional MI**, and
- the **bilinear difference-in-representation encoder versus a more general encoder**.

Metrics. While MI is an important signal for training, achieving high MI does not guarantee meaningful empowerment maximization. In a continuous setting without constraints or significant noise, prior work has shown that trivial behaviors can realize potentially unbounded MI (Ji et al., 2026). Thus, we additionally track environment-specific metrics to judge the quality of the learned policies. In Ant, we measure the distance of the ant to the center of the arena. In AntSoccer, we measure the distance of the ant to the soccer ball. In BuilderBench, we measure the distance of the gripper to the cubes. In Crafter, we measure the achievement count. These external metrics measure the quality of learned behaviors *independent* of the MI quantities themselves.

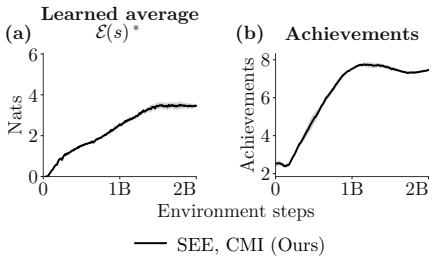
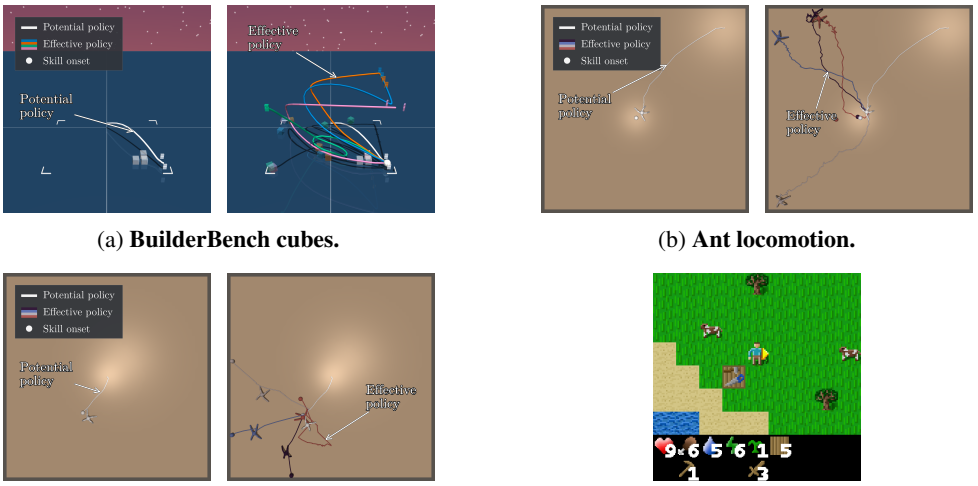


Figure 1: **The potential empowerment estimate increases with control in Crafter.** (a) The learned empowerment estimate increases consistently. (b) Simultaneously, the agent intrinsically learns to complete achievements in the environment *without access to external rewards*. Curves show mean ± standard error across three seeds.

5.1 Scaling and Baselines

Discrete Setting: Craftax-Classic. SEE scales to high-dimensional, discrete settings. In Craftax-Classic, the agent consistently unlocks $7-8$ out of 22 total achievements per rollout from 1×10^9 environment steps. These metrics exceed the best previously-reported success rates of intrinsic exploration methods in Crafter, which obtains ~ 5.5 achievements with the same training (Mohamed et al., 2026). The estimated empowerment also rises steadily (Fig. 1), tracking the development of interesting behavior. Implementation details and hyperparameters for Craftax-Classic are in Section B.2.

Continuous Settings: Ant & AntSoccer. SEE learns meaningful empowerment behavior in continuous settings that tracks increasing average empowerment estimates. If the empowerment is defined over all degrees-of-freedom ("Ant"), the policy π_{pot} learns to navigate to the *center region of the arena and stay there* (white, Fig. 2b) while the policies π_{eff} branch and cover the space (colors, Fig. 2b), following intuition that empowerment is high when the Ant can *do more* away from the walls of the arena. The baseline potential empowerment policies, in contrast, do not navigate to



(a) **BuilderBench cubes.** (b) **Ant locomotion.**

(c) **Ant soccer.** (d) **Craftax-Classic.**

Figure 2: **Empowerment-maximizing behavior across environments.** (a) In the BuilderBench task, a gripper interacts with blocks in an open-ended setting. The potential empowerment policy is visualized with the white rollout, navigating to the blocks and staying there. Then, the effective empowerment policies, or skills, knock the blocks in different directions (colorful lines and blocks). (b) In the first version of the AntSoccer task, an agent can be empowered without interacting with the ball. The potential empowerment policy (white) navigates to the central area of the arena, while the skills branch out from this point. (c) In the second version of the AntSoccer task, an agent can only be empowered if it controls the ball. The trained potential empowerment policy reaches the soccer ball, while skills kick the soccer ball in different directions. (d) In Craftax-Classic, the potential empowerment agent gathers materials, builds a crafting table, then builds tools using gathered materials. One seed per environment.

the center region and barely move at all. When the empowerment is defined only over the ball xy coordinates ("AntSoccer"), the policy π_{pot} navigates close to the ball (white, Fig. 2c) while the skill policies π_{eff} kick and dribble the ball in different directions (colors, Fig. 2c). The baseline potential empowerment policies, in contrast, do not consistently navigate close to the ball. When LatentGCE does happen to navigate close to the ball and kick the ball, the agents erroneously kick the ball to the very corner of the arena, where empowerment should be low.

Continuous Settings: BuilderBench. Finally, in BuilderBench, the potential empowerment policy sneaks behind the blocks, where it has easy access to the blocks, and stays there. The effective empowerment policies then knock the blocks in different directions (Fig. 3), and the arm learns to get closer to the blocks. In contrast, the baseline methods perturb the blocks, but fail to remain close to the blocks at the end of the potential empowerment rollout.

In summary, SEE scales to manipulation, locomotion, and open-ended (e.g. Crafter) environments, whereas prior empowerment-maximization methods struggle.

5.2 Ablating Design Decisions

Policy switching. During data collection, SEE switches from the potential empowerment to the effective empowerment policy. An alternative is to separately train potential empowerment and effective empowerment policies. Despite learning meaningful skills from the starting state, this ablation leads to inconsistent potential empowerment policies that do not reach the center of the arena in Ant nor consistently reach the blocks in BuilderBench. Meanwhile, our base SEE stably learns potential *and* effective empowerment policies in all of these environments.

Conditional MI vs. Joint MI Variants. Klyubin et al. (2005a) define empowerment as a conditional MI conditioned on a starting context state. However, modern MISL methods learn variational lower bounds on joint MIs (e.g. $I(Z; S, S')$) that do *not* condition on the starting context. We compare using conditional versus joint MI parameterizations of empowerment. Further details of how we

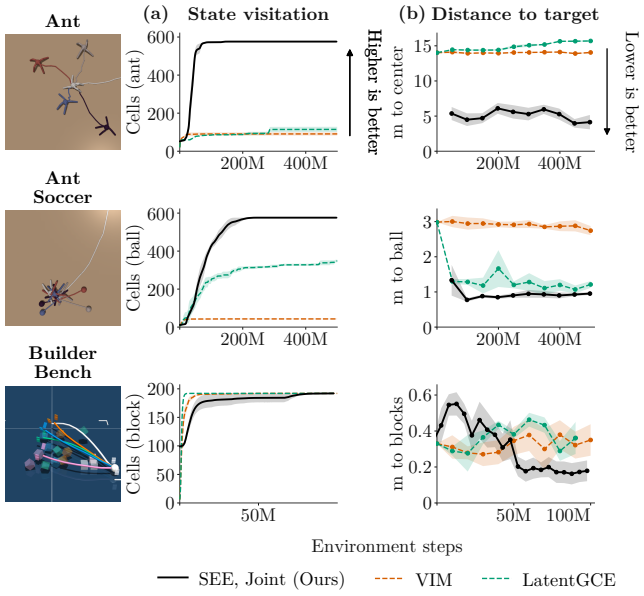


Figure 3: **Potential empowerment and control increases over training (continuous settings).** We compare SEE (Black) and baselines VIM (Orange) and LatentGCE (Green). (a) Only SEE visits diverse states across these environments, which is necessary to learn correct models and decoders of dynamics and skills. (b) SEE consistently learns potential empowerment policies that navigate to central regions in Ant, to the ball in AntSoccer, and to the blocks in BuilderBench. VIM is unstable and largely stationary in Ant and AntSoccer. LatentGCE is also stationary in Ant, and generally does not reach the blocks and soccer ball in the other environments as consistently as SEE. Five seeds for SEE; three per baseline.

implement the CMI version of SEE is in the Appendix A.2. Policies trained with the true conditional MI learn empowering behaviors in AntSoccer and BuilderBench. However, when empowerment is defined over many continuous degrees of freedom like in Ant, the CMI falls into the trap where the decoder and policies overfit to small perturbations, leading to high MI values without meaningful behavior. In Crafter, using the CMI leads to the best performance among the ablations and prevents the MI collapse observed when optimizing the joint MI.

Bilinear Difference in Representations vs. General Parameterization. Recent MISL works such as Park et al. (2023); Zheng et al. (2025); Park et al. (2026) utilize differences in representations $z^\top(\phi(s') - \phi(s))$ to parameterize classifiers (or similar objects) for skill learning. Replacing this parameterization with a more expressive logit significantly worsens the stability of training and the learned behaviors across environments, as the classifier can closely fit collected data and the policies "overfit" to sensitive classifiers.

5.3 Emergent Behaviors

Accruing and spending power. Across all environments, the learned switching policies accrue empowerment while executing the potential empowerment policy (Fig. 4, white traces). The policy gathers empowerment by walking to the center of the arena in Ant, getting close to the soccer ball in AntSoccer, getting close to the blocks in BuilderBench, and gathering resources and tools in Crafter. Then, the policies spend the empowerment when executing skills (Fig. 4, colorful traces), spreading out in the arena (Ant), kicking the ball (AntSoccer), and moving the blocks (BuilderBench).

Gathering resources and tool use. Resource-gathering and tool use enable empowerment-maximizing agents to exert control over the environment. In the Crafter environment, the trained potential empowerment agent learns to acquire many diverse materials (e.g. wood, saplings), such that its potential empowerment increases over the course of the episode. From that potentially-empowered state, the agent executes diverse skills such as building houses, mining, and using the gathered resources to craft new tools.

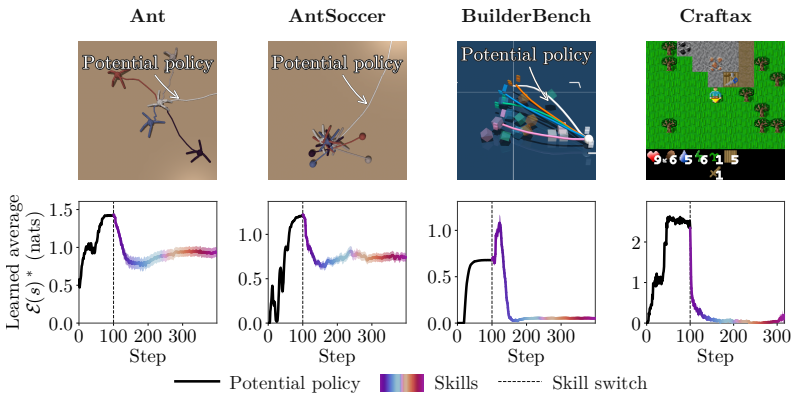


Figure 4: **Potential empowerment policies accrue empowerment, while skills spend it.** (Top row) Over the course of the trajectory, the potential empowerment policies "gather" empowerment, reaching empowered states. Correspondingly, the empowerment estimates increase over the course of the trajectory. (Bottom row) At the gray skill switch mark, executing a skill from the reached state "spends" the empowerment, reflecting the intuition that potential empowerment policies collect power and effective empowerment spends power. Traces use one seed and 30 skills per environment.

Modifying the environment. Empowered agents learn to perturb their environment to maximize future control. This phenomena has been documented in prior works (Salge et al., 2014a) in simple tabular settings. In the scaled up Crafter setting, we observe more complex "building" behaviors, such as learning to construct a house when defending against enemies.

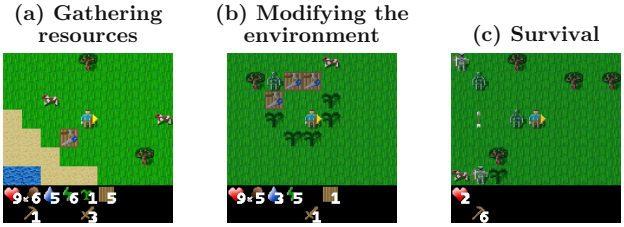


Figure 5: **Empowerment maximization has interesting emergent behaviors.** (a) In Crafter, the empowered agent quickly accumulates many resources and builds a tool (crafting table) necessary to craft other tools in the game. (b) Early in training, the agent learns to build a "house" consisting of plants and crafting tables. (c) The agent also naturally learns to survive, avoiding and killing enemies. One seed per example.

Survival. In death, an agent no longer has any control and, thus, has no empowerment. Therefore, agents maximizing empowerment should learn survival skills. In Crafter, we see that empowerment-maximizing agents naturally learn survival skills like killing and dodging enemies.

6 Discussion

Despite the conceptual appeal of empowerment as a tool for understanding natural agents and building artificial agents, its seeming computational intractability has caused prior work to resort to approximations that do not learn one policy that "keeps its options open" (Klyubin et al., 2008). In providing a conceptual link between skill learning methods and empowerment, we provide a method that maximizes potential and effective empowerment, two policies that measure controllability in distinct ways. We present key practical considerations to learn empowerment maximization policies in high-DoF settings. Future work should investigate simplified parameterizations of empowerment maximization that can resolve practical considerations detailed within our method, at scale. Furthermore, our work highlights the sensitivity of empowerment maximization to representations of the world, with open questions on the interplay between representations of the world and empowerment maximization.

AI USE STATEMENT

We used generative AI tools to assist with implementing experiment code, preparing and revising figures, minor writing edits, assembling appendix details, and manuscript formatting. We did not use AI for the methods development, theoretical results, or any major writing. The authors take responsibility for the final content of this work.

REPRODUCIBILITY STATEMENT

We provide algorithm pseudocode and implementation details in the appendix. Code, videos, and additional figures are available at `tinyurl.com/2a7nzbrb`.

REFERENCES

David Abel, Michael Bowling, André Barreto, Will Dabney, Shi Dong, Steven Hansen, Anna Harutyunyan, Khimya Khetarpal, Clare Lyle, Razvan Pascanu, et al. Plasticity as the Mirror of Empowerment. *Neural Information Processing Systems*, 38:164340–164374, 2026.

David Barber and Felix Agakov. The IM Algorithm: A Variational Approach to Information Maximization. *Neural Information Processing Systems*, volume 16, 2003.

Marc Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. Unifying Count-Based Exploration and Intrinsic Motivation. *Neural Information Processing Systems*, 29, 2016.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by Random Network Distillation. *International Conference on Learning Representations*, 2019.

Philippe Capdepuy. Informational Principles of Perception-Action Loops and Collective Behaviours. 2011.

Fei Dai and Alison Gopnik. Understand Intrinsic Motivation in Causal Learning Through Exploration. *Annual Meeting of the Cognitive Science Society*, volume 47, 2025.

Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity Is All You Need: Learning Skills Without a Reward Function. *International Conference on Learning Representations*, 2018.

Benjamin Eysenbach, Ruslan Salakhutdinov, and Sergey Levine. The Information Geometry of Unsupervised Reinforcement Learning. *International Conference on Learning Representations*, 2021.

Matteo Gallici, Mattie Fellows, Benjamin Ellis, Bartomeu Pou, Ivan Masmitja, Jakob Foerster, and Mario Martin. Simplifying Deep Temporal Difference Learning. *International Conference on Learning Representations*, 2025.

Raj Ghugare, Roger Creus Castanyer, Catherine Ji, Kathryn Wantlin, Jin Schofield, Karthik Narasimhan, and Benjamin Eysenbach. BuilderBench: The Building Blocks of Intelligent Agents. arXiv:2510.06288, 2026.

Alison Gopnik. Empowerment as Causal Learning, Causal Learning as Empowerment: A Bridge Between Bayesian Causal Hypothesis Testing and Reinforcement Learning. 2024.

Karol Gregor, Danilo Jimenez Rezende, and Daan Wierstra. Variational Intrinsic Control. *International Conference on Learning Representations*, 2017.

Lucas Gruaz, Alireza Modirshanechi, Sophia Becker, and Johanni Brea. Merits of Curiosity: A Simulation Study. 2024.

Rein Houthooft, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. Vime: Variational Information Maximizing Exploration. *Neural Information Processing Systems*, 29, 2016.

Khurram Javed and Richard S Sutton. The Big World Hypothesis and Its Ramifications for Artificial Intelligence. *Finding the Frame: An RLC Workshop for Examining Conceptual Frameworks*, 2024.

Catherine Ji, Vivek Myers, Sergey Levine, and Benjamin Eysenbach. The Geometry of Empowerment. 2026.

Maximilian Karl, Maximilian Soelch, Philip Becker-Ehmck, Djalel Benbouzid, Patrick van der Smagt, and Justin Bayer. Unsupervised Real-Time Control through Variational Empowerment. arXiv:1710.05101, 2017.

Alexander S Klyubin, Daniel Polani, and Chrystopher L Nehaniv. All Else Being Equal Be Empowered. *European Conference on Artificial Life*, pp. 744–753, 2005a.

Alexander S Klyubin, Daniel Polani, and Chrystopher L Nehaniv. Keep Your Options Open: An Information-Based Driving Principle for Sensorimotor Systems. *PloS One*, 3(12):e4018, 2008.

A.S. Klyubin, D. Polani, and C.L. Nehaniv. Empowerment: A Universal Agent-Centric Measure of Control. *IEEE Congress on Evolutionary Computation*, volume 1, pp. 128–135 Vol.1, 2005b.

Andrew Levy, Alessandro G Allievi, and George Konidaris. Representation Learning and Skill Discovery With Empowerment. *Reinforcement Learning Conference*, 2025.

Aly Lidayan, Yuqing Du, Eliza Kosoy, Maria Rufova, Pieter Abbeel, and Alison Gopnik. Intrinsically-Motivated Humans and Agents in Open-World Exploration. arXiv:2503.23631, 2025.

Zhuang Ma and Michael Collins. Noise Contrastive Estimation and Negative Sampling for Conditional Models: Consistency and Statistical Efficiency. *Empirical Methods in Natural Language Processing*, pp. 3698–3707, 2018.

Michael Matthews, Michael Beukman, Benjamin Ellis, Mikayel Samvelyan, Matthew Jackson, Samuel Coward, and Jakob Foerster. Craftax: A Lightning-Fast Benchmark for Open-Ended Reinforcement Learning. arXiv:2402.16801, 2024.

Faisal Mohamed, Catherine Ji, Benjamin Eysenbach, and Glen Berseth. Temporal Representations for Exploration: Learning Complex Exploratory Behavior Without Extrinsic Rewards. 2026.

Shakir Mohamed and Danilo Jimenez Rezende. Variational Information Maximisation for Intrinsically Motivated Reinforcement Learning. *Neural Information Processing Systems*, 28, 2015.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation Learning With Contrastive Predictive Coding. arXiv:1807.03748, 2018.

Jongchan Park, Seungjun Oh, Seungho Baek, and Yusung Kim. Learning Generalizable Skill Policy With Data-Efficient Unsupervised RL. aXiv:2607.00392, 2026.

Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking Offline Goal-Conditioned RL. *International Conference on Learning Representations*, 2025.

Seohong Park, Oleh Rybkin, and Sergey Levine. Metra: Scalable Unsupervised RL With Metric-Aware Abstraction. *International Conference on Learning Representations*, 2023.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-Driven Exploration by Self-Supervised Prediction. *International Conference on Machine Learning*, pp. 2778–2787, 2017.

Martin L Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2014.

Christoph Salge, Cornelius Glackin, and Daniel Polani. Changing the Environment Based on Empowerment as Intrinsic Motivation. *Entropy*, 16(5):2789–2819, 2014a.

Christoph Salge, Cornelius Glackin, and Daniel Polani. Empowerment–an Introduction. *Guided Self-Organization: Inception*, pp. 67–114. Springer, 2014b.

Archit Sharma, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. Dynamics-Aware Unsupervised Discovery of Skills. *International Conference on Learning Representations*, 2020.

Stas Tiomkin, Christoph Salge, and Daniel Polani. Process Empowerment for Robust Intrinsic Motivation. *Journal of Physics: Complexity*, 6(3):035011, 2025.

Alexander Matt Turner, Logan Smith, Rohin Shah, Andrew Critch, and Prasad Tadepalli. Optimal policies tend to seek power. *arXiv preprint arXiv:1912.01683*, 2019.

Claire Yang, Claire Jie Zhang, Maya Cakmak, and Max Kleiman-Weiner. When Assisting One Disempowers Another. arXiv:2511.04177, 2026.

Eunice Yiu, Kelsey Allen, Shiry Ginosar, and Alison Gopnik. Empowerment Gain and Causal Model Construction: Children and Adults Are Sensitive to Controllability and Variability in Their Causal Interventions. arXiv:2512.08230, 2026.

Ruihan Zhao, Kevin Lu, Pieter Abbeel, and Stas Tiomkin. Efficient Empowerment Estimation for Unsupervised Stabilization. aXiv:2007.07356, 2021.

Chongyi Zheng, Jens Tuyls, Joanne Peng, and Benjamin Eysenbach. Can a MISL Fly? Analysis and Ingredients for Mutual Information Skill Learning. *ICLR 2025*, 2025.

A Algorithm and Implementation Details

Below, we include key algorithm and implementation details. While we describe the Joint MI classifier form of the SEE for simplicity, we detail the Conditional MI variant of SEE later in Section A.2. Note that when the skill is independent of the starting state, these classifiers are equivalent.

A.1 Training the classifier

SEE utilizes MISL methods to lower bound the pointwise potential empowerment. We take an approach similar to CSF (Zheng et al., 2025), employing noise contrastive estimation to learn the skill classifier, which provides a variational lower-bound on the MI.

Without loss of generality, assume that the skills z are drawn from vMF distribution, so $p(z) = \mathrm{Unif}(\mathbb{S}^{d-1})$. Let $f_\phi(s, s_+, z)$ denote a parameterized logit over a state s, future state s_+, and skill z, constrained to take the form

$$f_\phi(s, s_+, z) = \frac{1}{\tau} z^\top (\phi(s_+) - \phi(s))$$

which we refer to as a bilinear difference-of-representations parameterization, with temperature $\tau = 0.1$. This parameterization of the logit appears in recent SOTA MISL literature and has been shown to train persistent skill behavior that spread over latent space; we refer readers to these references for more information (Zheng et al., 2025; Park et al., 2023; 2026).

Then, given access to a dataset $\mathcal{D}$ of starting states, skills, and outcomes (s, s_+, z), where outcomes $s_+ \sim p^\pi(\cdot \mid s, z)$ are drawn from the DSOM, InfoNCE trains a classifier to distinguish whether skills are drawn from the joint distribution $p(z, s, s_+)$ or the product of marginals $p(z)p(s, s_+)$:

$$\text{InfoNCE Loss:} \quad \mathcal{L}(\phi) = \mathbb{E}_{(s,s_+,z)\sim p(s,s_+,z)} \left[\log \frac{\exp(f_\phi(s, s_+, z))}{\mathbb{E}_{z_- \sim p(z)} \exp\left(f_\phi(s, s_+, z_-)\right)} \right].$$

We denote skills drawn from the marginal $p(z)$ by z_- for clarity. At optimum, the logit recovers the true classifier

$$f^*_\phi(s, s_+, z) = \log \frac{p(z \mid s, s_+)}{p(z)} + c_\phi(s, s_+)$$

where $c_\phi(s, s_+)$ denotes the log partition function

$$c_\phi(s, s_+) \triangleq \log \mathbb{E}_{p(z)} \left[\exp\big(f^*_\phi(s, s_+, z)\big) \right].$$

The learned logit gives a variational lower bound on the MI $I(Z; S, S_+)$:

$$\mathbb{E}_{p(z,s,s_+)} \big[f_\phi(s, s_+, z) - c_\phi(s, s_+) \big] \leq I(Z; S, S_+),$$

which becomes tight at the optimum (Ma and Collins, 2018).

Adding an Information Bottleneck. High MI *does not directly imply interesting behavior in continuous settings*. Prior works note that without any type of constraint on the information maximization problem, empowerment is ill-defined in continuous settings and can be infinite (Salge et al., 2014b). In our case, a fully-general skill decoder trained over the transitions can give arbitrarily large empowerment estimates, even for skills that barely change the state.

It is critical to apply a form of information bottleneck from $S_0, S_+ \to Z$ when computing intrinsic rewards in the continuous setting for the inner skill-learning optimization problem. For simplicity, we uniformly add Gaussian noise to the state when training the classifier. This noising forces the classifier to identify skills from perturbations in the state beyond this noise scale. Concretely, we train $f_\phi(s + \epsilon, s_+ + \epsilon_+, z)$, redrawing the noise $\epsilon, \epsilon_+ \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2 I)$ at each update. Noise scales for each environment are given in Section B.

A.2 Data Collection & Conditional MI

During data collection, SEE switches from the potential empowerment at a switch state s_0 to the effective empowerment policy. The purpose of this switch is to ensure that the method learns skills precisely at reachable locations. During rollout, we take the skill distribution $p(z \mid s_0) = p(z)$ to be uniform.

The difference between the conditional MI versus the joint MI classifier variants of SEE happens at the level of data collection. If skills are sampled uniformly-at-random from the initial state s_0, the MI $I(Z; S_0)$ should equal 0. Thus, a simple way to optimize the conditional MI during the inner MISL optimization is to roll out multiple skills sampled from the uniform marginal $p(z)$ from the same initial state and train the classifier as usual. Concretely, for a switch state s_0, draw *several* skills $z \sim p(z)$ and roll out, "branching" these skills from this switch state. Then, the distributions $p(z \mid s_0)$ and $p(z)$ are equivalent, and the joint MI and conditional MI are equivalent:

$$I(Z; S_0, S_+) = I(Z; S_0) + I(Z; S_+ \mid S_0) = I(Z; S_+ \mid S_0).$$

A.3 Training empowerment value functions and policies

A.3.1 Inner Optimization: Skill Learning/Effective Empowerment

We consider continuous skill spaces for all of our learned skills.

Markovian Skill Policies & Critics in Continuous Settings. The skill policies and value functions for the inner optimization problem condition on the start state, the current state, and a skill (Section 4.1). As a result, the skill policy is a function of (s, s_0, z) and the skill critic is a function of (s, s_0, a, z). Meanwhile, the potential policy simply uses the pointwise estimate of the potential empowerment as a state-wise reward, so the potential policy and critics take as input s and (s, a), respectively. Each SAC critic consists of two independent multilayer perceptrons, each with two 512-unit hidden layers, GELU activations, layer normalization, and a scalar Q-value output.

Nonmarkovian Skill Policies & Critics in Crafter. In the case of a nonmarkovian policy as used in the Crafter environment, which is partially-observed, the policies and PQN (Gallici et al., 2025) critics take in an additional recurrent state. Following (Gallici et al., 2025), each Q-network uses a 512-unit fully connected layer with layer normalization and ReLU, followed by a 512-unit LSTM and a linear output giving one Q-value per action. The LSTM receives the encoded inputs and previous action. The hidden and cell states carry information from earlier steps.

Skill Rewards. As mentioned in Section 4.1, setting the reward to the normalized classifier score provides a lower bound on the MI in expectation, which we show later in Proposition 3. In practice, for the inner optimization problem, MISL methods like CSF (Zheng et al., 2025) utilize the *unnormalized* classifier as the reward, because the latter partition term has a skill-agnostic, anti-exploration effect. In our case, while we desire the exploration effects of the unnormalized reward, for the goal of potential empowerment maximization, we need the inner optimization to actually lower bound the pointwise potential empowerment. Thus, at the start of training, we begin with the unnormalized reward to get exploration ($\lambda = 0$), then turn on the anti-exploration term over training ($\lambda = 1$) so that, at the end, the method estimates pointwise empowerment: The effective-policy reward is

$$r_{\text{eff}}(s, a, s'; s_0, z) = f_\phi(s_0, s', z) - \lambda \log \mathbb{E}_{\tilde{z} \sim p(z)}[\exp f_\phi(s_0, s', \tilde{z})]\,.$$

Specifically, SEE sets $\lambda = 0$ for the first 30% of training, increases it linearly to 1 over the next 50%, and keeps it at 1 for the final 20%. In experiments, we see that this annealing does not significantly change training in continuous environments. However, in Craftax-Classic, this annealing is critical to get the method to sufficiently explore to get stronger empowerment estimates.

A.3.2 Outer Optimization: Potential Policy and Critic Optimization

The potential empowerment critic simply takes the Monte-Carlo estimate of the expected skill critic over 8 uniformly-sampled skills as the reward:

$$r_{\text{pot}} = \frac{1}{8} \sum_{k=1}^{8} Q(s, a_k; b, z_k), \quad z_k \sim p, \quad a_k \sim \pi_{\text{eff}}(\cdot \mid s, z_k; b).$$

In expectation, this estimate is a lower bound on the pointwise potential empowerment (see Eq. (9)).

Separating Actor and Critic Buffers from the Classifier Buffer. All the critics used in this method can theoretically work in off-policy settings. However, the skill classifier used in the inner optimization cannot similarly train on an extensively off-policy replay buffer – indeed, early on in training, we should expect that the classifier has very low accuracy, which would underestimate the *current* potential empowerment. Thus, we maintain separate buffers for actor-critic training and classifier training, where the classifier buffer is generally much smaller than that of the actor-critic buffers. Exact parameters are in Section B.

A.4 Empowerment maximization pseudocode

Algorithm 1: SEE

Require: Skill prior $p(z)$, episode length T, switch time M, iterations N

Initialize potential policy π_{pot}, skill policy π_{eff}, their critics, classifier f_ϕ, and buffers.
for $n = 1, \ldots, N$ **do**
  Collect: Run π_{pot} for M steps from reset; record the switch state s_0 (Section 4.2).
  Draw $z \sim p(z)$ and run $\pi_{\text{eff}}(\cdot \mid s, z; s_0)$ for $T - M$ steps, holding z, s_0 fixed.
  Store the two-stage rollout for policy and critic updates.
  Fit classifier: Train f_ϕ with InfoNCE on states and discounted future states from skill execution (Joint MI), or branches with fresh skills $z \sim p(z)$ from copied starting states (CMI; Section A.2).
  Inner optimization: Update π_{eff} and its critics using r_{eff} (Sections A.3 and 4.1).
  Estimate empowerment: Average skill values over eight draws $z \sim p(z)$ (Eq. (9)).
  Outer optimization: Use this estimate as r_{pot} to update π_{pot} and its critic (Section 4). SAC uses replay; PQN uses the two-stage rollout; CMI branches train only f_ϕ.
return $\pi_{\text{eff}}, \pi_{\text{pot}}$

B Experimental setup and evaluation

Continuous Env Evaluation. Like in the data collection during training, the evaluation rollouts run the potential empowerment policies for the first 100 steps. Then, the rollout draws a skill $z \sim p(\cdot)$, then executes the effective empowerment policy by sampling a skill, rolling out until termination or truncation. Evaluations use 240 episodes for Ant/AntSoccer and 128 for BuilderBench.

The potential empowerment target distance is measured *at the switch point*/at the end of the potential empowerment phase of the rollout. Meanwhile, the skill displacements are measured from the switch point to the end of the skill rollout. Visitation curves count locations visited during training of the ant (Ant), ball (AntSoccer), and any of the blocks (BuidlerBench) during training. The locations are discretized cells of length 1 m cells for the ant or ball and 5 cm cells for the blocks.

Craftax Classic Evaluation. We use the externally-defined achievement count for evaluation in Craftax-Classic, measuring the total achievements accrued over the course of the two-phase rollout.

Curves show the mean $\pm$ standard error across seeds. Training and Craftax curves use per-seed exponential smoothing ($\alpha = 0.05$) before averaging. Distance, displacement, and phase-achievement curves are unsmoothed.

B.1 Continuous Experiment Details

In the figures, Ant denotes the full-state classifier (42 features), while AntSoccer denotes the classifier using only ball x, y; BuilderBench uses the three blocks' positions. Visitation counts distinct planar cells reached during training: 1 m cells for the ant or ball and 5 cm cells pooled across the three blocks.

B.1.1 ANTSOCCER, FULL OBS

Hyperparameters are in Table 1. Training curves are in Fig. 3.

Table 1: Hyperparameters in AntSoccer, full obs (SEE, Joint MI (Ours); quantitative curves).

Parameters	**Value**
Environment	AntSoccer arena (navigate, single task)
Agent	potential empowerment (SAC)
State dimension	42
Action dimension	8
Ball observations only	False
Center ball on reset	False
Agent start center fraction	0
Observation noise std (classifier bottleneck)	0.05
Skill dimension d	2
Skill prior	uniform unit sphere in $\mathbb{R}^d$
Potential policy steps	100
Max episode steps	400
Skill policy steps	300
Training seeds	5
Online env steps per seed	500,000,000
Actor/Critic Replay Buffer	10,000,000 transitions
Encoder Buffer	3,334 episodes (1,000,200 transitions)
Future-sampling discount	0.99
Encoder reference	current state
Reward partition weight λ	0 initially; linear $0 \to 1$ from 150M to 400M env steps; 1 thereafter
Target entropy	0 (learned temperature, init 1)
Learning rate	3e-4
Batch size	1,024
SAC replay samples per collected transition	4
Encoder samples per collected transition	1
Critic target update rate	0.02
Discount γ	0.99
Contrastive temperature	0.1
Separate skill-0 actor and critic	True
Actor / critic / encoder hidden dims	[512, 512]
Layer norm (actor / critic / encoder)	False / True / True
Potential-value skill samples	8

B.1.2 ANTSOCCER, BALL OBS ONLY

Hyperparameters are in Table 2. Training curves are in Fig. 3.

Table 2: Hyperparameters in AntSoccer, ball obs only (SEE, Joint MI (Ours); quantitative curves).

Parameters	**Value**
Environment	AntSoccer arena (navigate, single task)
Agent	potential empowerment (SAC)
State dimension	42
Action dimension	8
Ball observations only	True
Center ball on reset	True
Agent start center fraction	0.8
Observation noise std (classifier bottleneck)	0.05

Continued on next page

Table 2 (continued)

Parameters	Value
Skill dimension d	2
Skill prior	uniform unit sphere in $\mathbb{R}^d$
Potential policy steps	100
Max episode steps	400
Skill policy steps	300
Training seeds	5
Online env steps per seed	500,000,000
Actor/Critic Replay Buffer	10,000,000 transitions
Encoder Buffer	3,334 episodes (1,000,200 transitions)
Future-sampling discount	0.99
Encoder reference	current state
Reward partition weight λ	0 initially; linear $0 \to 1$ from 150M to 400M env steps; 1 thereafter
Target entropy	0 (learned temperature, init 1)
Learning rate	3e-4
Batch size	1,024
SAC replay samples per collected transition	4
Encoder samples per collected transition	1
Critic target update rate	0.02
Discount γ	0.99
Contrastive temperature	0.1
Separate skill-0 actor and critic	True
Actor / critic / encoder hidden dims	[512, 512]
Layer norm (actor / critic / encoder)	False / True / True
Potential-value skill samples	8

B.1.3 BUILDERBENCH

Hyperparameters are in Table 3. The figures of MI over environment steps and displacement over environment steps are in Fig. 3.

Table 3: Hyperparameters in BuilderBench (SEE, Joint MI (Ours); quantitative curves).

Parameters	Value
Environment	BuilderBench (3 cubes, corner cluster)
Agent	potential empowerment (SAC)
State dimension	51
Action dimension	5
Cube observations only	True
Observation noise std (classifier bottleneck)	0.01
Skill dimension d	9
Skill prior	uniform unit sphere in $\mathbb{R}^d$
Potential policy steps	100
Max episode steps	400
Skill policy steps	300
Training seeds	5
Online env steps per seed	100,000,000
Actor/Critic Replay Buffer	6,000,000 transitions
Encoder Buffer	3,334 episodes (1,000,200 transitions)
Future-sampling discount	0.99
Encoder reference	current state
Reward partition weight λ	0 initially; linear $0 \to 1$ from 30M to 80M env steps; 1 thereafter
Target entropy	-12.5 (learned temperature, init 1)

Continued on next page

Table 3 (continued)

Parameters	Value
Learning rate (actor, critic, temperature, encoder)	3e-4
Batch size	256
SAC replay samples per collected transition	4
Encoder samples per collected transition	1
Critic target update rate	0.02
Discount γ	0.99
Contrastive temperature	0.1
Separate skill-0 actor and critic	True
Actor / critic / encoder hidden dims	[512, 512]
Layer norm (actor / critic / encoder)	False / True / True
Potential-value skill samples	8

B.2 Discrete Experiment Details

B.2.1 Craftax-Classic

Hyperparameters are in Table 4. Fig. 6 shows one two-phase episode. We use PQN (Gallici et al., 2025) rather than SAC to utilize the GPU acceleration in the Craftax environment.

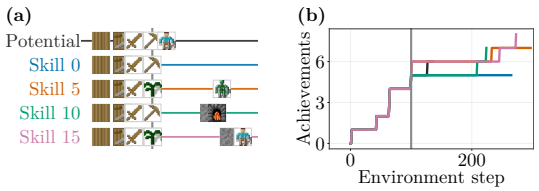


Figure 6: **Achievements in a single Craftax-Classic two-phase episode.** Icons indicate when achievements first occur. (a) The black line shows the achievements of the shared potential phase up to the skill switch (vertical gray line). In this rollout, these achievements include collecting wood, building a crafting a table, then crafting a wooden sword. The colorful skill rows show the achievements rolled out from the switch point sharing the same starting state, including sleeping/waking up, building a pickaxe, planting a tree, collecting stone, building a furnace, and defeating an enemy. (b) The lines indicate the cumulative achievement counts over the rollout, achieved without the use of any extrinsic reward. One seed.

Table 4: Hyperparameters in Craftax-Classic (SEE, CMI (Ours); quantitative curves).

Parameters	Value
Environment	Craftax-Classic (symbolic)
Agent	potential empowerment (Potential-PQN)
Observation dimension	1345
Number of discrete actions	17
Encoder observations	inventory only
Inventory feature dimension	12
Encoder data	CMI branches from copied states
Observation noise std (classifier bottleneck)	0
Encoder norm cap c	disabled
Future-sampling discount	0.95
Logit temperature τ	0.1 (fixed)
Encoder learning rate	1e-3
Encoder batch size	4,096
Encoder hidden dims	[512, 512]
Encoder Buffer	10,000,000 branch transitions
Skill dimension d	16
Skill prior	uniform unit sphere in $\mathbb{R}^d$
Potential policy steps	100
Max episode steps	1000

Continued on next page

Table 4 (continued)

Parameters	Value
Skill policy steps	900
Training seeds	3
Encoder branch horizon	900
Extra branch steps per live step	0.9 (900 additional simulator steps per 1,000 live training steps)
Online env steps per seed (live)	2,000,000,000
Reward	alignment minus λ log-partition
Skill-0 reward	normalized potential estimate (Eq. (9))
Potential-value skill samples	8
Achieved-skill relabeling	False
Hindsight anchor relabeling	False
Reward partition weight λ	0 initially; linear $0 \to 1$ from 600M to 1.6B live env steps; 1 thereafter
Separate skill-0 and skill Q-networks	True
Learning rate	3e-4 (linear decay to 0)
PQN rollout batch	131,072 transitions
Minibatches	4
Sequences per minibatch	256 (128 steps each)
Update epochs	4
Encoder updates per rollout batch	4
Exploration	ϵ-greedy: $1 \to 0.005$ over 100M live env steps
Discount γ	0.95
PQN λ	0.5
Max gradient norm	0.5
Q-network hidden dims	MLP [512]; LSTM 512
Layer norm	True
Q-network input normalization	Batch renormalization

C BASELINES

Unlike our method, both VIM (Mohamed and Jimenez Rezende, 2015) and LatentGCE (Zhao et al., 2021) consider open-loop definitions of empowerment, where A_0^{H-1} is shorthand for the action sequence $(a_0, \ldots, a_{H-1})$:

$$\mathcal{E}_{\text{VIM}}(s_0) \triangleq \max_{\pi \in \Pi_{\mathcal{Z}}} I^{\pi}(A_0^{H-1}; S_H \mid S_0 = s_0). \tag{10}$$

VIM. VIM utilizes a variational lower bound on the empowerment with a learned decoder $q_\theta(a_1^H \mid s_0, s_H)$, a generative model for the action sequence. VIM fits this decoder using a standard cross-entropy loss. Then, by definition, learning a source distribution $\omega(a_0^{H-1} \mid s_0)$ that maximizes this variational lower bound gives a lower bound on the true open-loop potential empowerment. We follow the autoregressive source and decoder in Mohamed and Jimenez Rezende (2015, Sec. 4.2), using tanh-Gaussians for our continuous actions.

Fixing the estimated decoder $q_\theta(a_1^H \mid s_0, s_H)$, at optimum, the source distribution $\omega(a_0^{H-1} \mid s_0)$ takes the form

$$\omega^\star(a_0^{H-1} \mid s_0) \propto \exp(\mathbb{E}_{p(s_H \mid s_0, a_0^{H-1})}[\log q_\theta(a_0^{H-1} \mid s_0, s_H)]).$$

However, computing the partition function/normalizer is generally intractable. Instead, VIM regresses a function $f_\phi(s_0, a_0^{H-1}) \triangleq \log h_\phi(a_0^{H-1} \mid s) + \psi_\phi(s_0)$ onto the energy $\mathbb{E}_{p(s_H \mid s_0, a_0^{H-1})}[\log q_\theta(a_0^{H-1} \mid s_0, s_H)]$, where h_ϕ is normalized. At optimum, the function $\psi_\phi(s_0)$ converges to an approximation of the open-loop potential empowerment by the standard variational Gibbs identity. We refer the reader to Mohamed and Jimenez Rezende (2015) for more details.

LatentGCE. The next baseline we consider is LatentGCE (Zhao et al., 2021). Unlike VIM, LatentGCE learns a generative, forward-in-time model to estimate empowerment. LatentGCE

observes that learning a latent model of the environment that evolves according to linear-quadratic regulator (LQR) dynamics immediately gives a closed-form solution for the open-loop potential empowerment. This approach sidesteps the challenges of optimizing a source distribution by directly learning a generative model with a simple LQR form.

C.1 Validating Empowerment Fields in Simple Continuous Benchmarks

We compare the learned empowerment fields in two simple, didactic continuous environments of a pendulum and ball-in-box discussed in LatentGCE. The learned empowerment fields are in Fig. 7. Our implementations qualitatively reproduce the behavior reported in Zhao et al. (2021), where LatentGCE succeeds to find the high empowerment region while VIM does not.

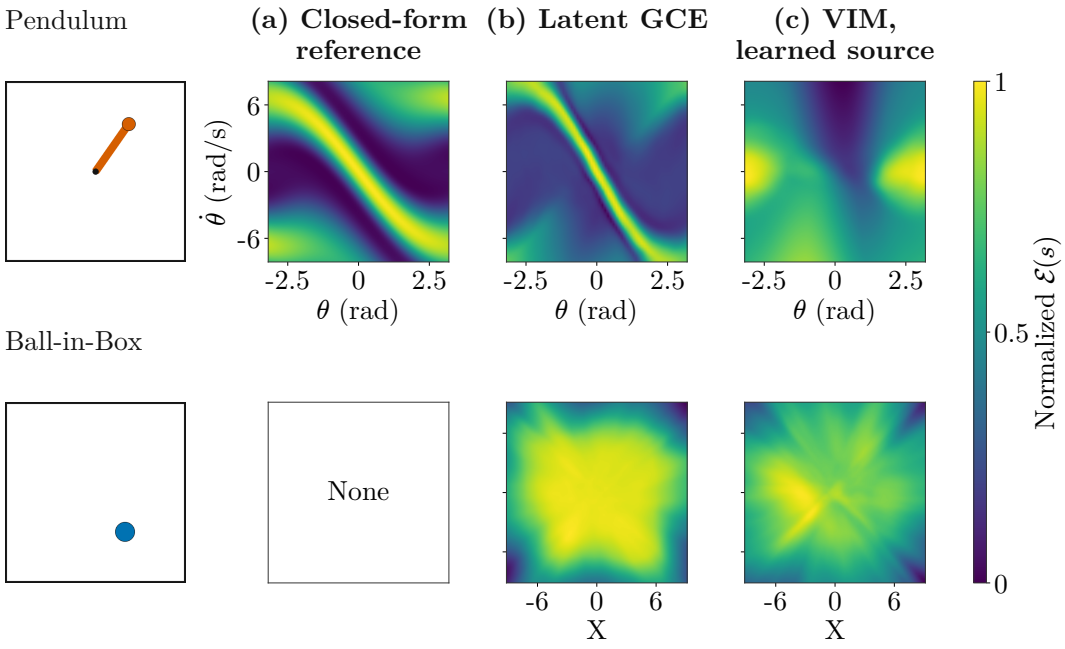


Figure 7: **Empowerment fields on low-dimensional benchmarks.** For the canonical environments of a pendulum and a ball in a box with hard walls, various empowerment maximization methods predict (normalized) empowerment fields over state space. (a) A linearized reference reveals that the potential empowerment is maximized when the pendulum points straight up. For the Ball in Box, the potential empowerment is maximal at the center of the box. Thus, a potential empowerment policy over a long effective horizon should seek a straight-up state in the pendulum and the central state for Ball in Box. (b) The empowerment field predicted by LatentGCE method correctly identifies these potential empowerment states. (c) VIM does not recover these high empowerment regions. One seed per learned field.

C.2 Implementation in the Main Evaluation Environments

We then apply VIM and LatentGCE to Ant, AntSoccer, and BuilderBench following the referenced manuscript implementations for pixel-based settings. We use a horizon of $H = 30$. Both VIM and LatentGCE struggle, at least partially due to the open-loop empowerment parameterization over long action sequences.

Table 5: VIM and LatentGCE hyperparameters for Ant, AntSoccer, and BuilderBench (Sections B and 5). Budgets include controller and probe transitions; each method uses three seeds per environment.

Parameter	VIM	LatentGCE
Training budget (Ant / AntSoccer / BuilderBench)	500M / 500M / 100M	
Episode length / probe horizon H	400 / 30 steps	

Table 5: VIM and LatentGCE hyperparameters (continued).

Parameter	VIM	LatentGCE
Controller observations (Ant / AntSoccer / BuilderBench)	42 / 42 / 51 features	
Estimator outcomes (Ant / AntSoccer / BuilderBench)	Full state (42) / ball x, y (2) / block positions (9)	
Parallel environments / rollout length	256 / 80 steps	
Probe starts per update	256; one uniformly sampled rollout time per environment	
Estimator replay capacity / minibatch	10,000 tuples / 1,024	
Estimator passes over replay per update	10	
PPO controller (shared)		
Actor / critic hidden layers	[256, 256], tanh	
Optimizer / learning rate / Adam ϵ	Adam / 10^{-3} / 10^{-5}	
Discount γ / GAE λ	0.99 / 0.95	
Update epochs / minibatches	4 / 4 (5,120 transitions each)	
PPO clipping / max. gradient norm	0.2 / 0.5	
Entropy / value loss coefficients	0.01 / 0.5	
Action distribution	Tanh-Gaussian, actions in $[-1, 1]^{d_a}$	
Observation normalization	None	
Empowerment estimator		
Optimizer / Adam ϵ	Adam / 10^{-8}	
Learning rate	3×10^{-4}	10^{-4}
Hidden layers	Source/decoder: [512, 512, 512]	Encoders/channel: [512, 512, 512]
Additional hidden layers	Scalar ψ: [256, 256, 256]	Reconstruction decoders: [512, 512]
Activation	ReLU	ReLU
State / action latent dimension	—	32 / 32
Source initial standard deviation	1	—
Source/decoder log-standard-deviation bounds	$[-5, 2]$	—
VIM coefficient β	1	—
Channel power budget	—	1
Water-filling tolerance / iteration limit	—	10^{-3} / 10,000
Latent prediction / state reconstruction loss weights	—	1 / 500
Action reconstruction / latent norm loss weights	—	10 / 1

D Ablation Details

Here, we include additional ablations and ablation details. The corresponding figures are in Fig. 9 and Fig. 10.

D.1 Policy switching.

The first design decision we ablate is policy switching. An alternative is to separately train potential empowerment and effective empowerment policies:

Ours: Policy switching from potential to effective during data collection.
Ablation: Training skills and potential policy from the the same starting state, without skill switching.

Without switching, the method cannot consistently learn meaningful potential empowerment policies in Ant and BuilderBench. Meanwhile, our base SEE uses episodic rollouts and successfully learns potential and effective empowerment policies in all environments.

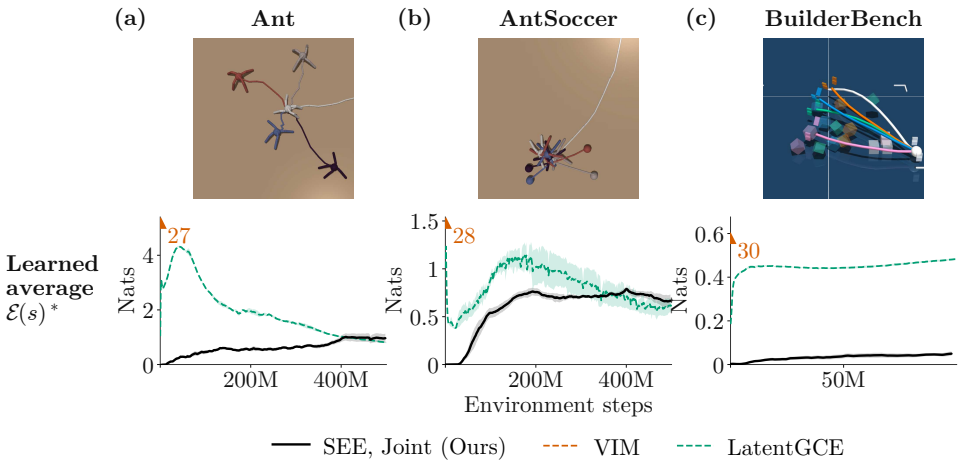


Figure 8: **Learned empowerment in continuous settings.** SEE (Black) empowerment values gradually increase over time. Meanwhile, VIM (Orange) and LatentGCE (Green) empowerment values are unstable. *As a note, learned estimates use different objectives and regularization, so their magnitudes are not directly comparable across methods. Five seeds for SEE; three per baseline.

D.2 Conditional versus full MI.

CMI uses the branching rollout scheme detailed in Sec A.2. All other training stays fixed. We observe that using CMI leads to hurt training, particularly in the high-dimensional, continuous setting of Ant where it is easy for a properly context-conditioned classifier to overfit. Meanwhile, using the Joint MI in Craftax-Classic leads to collapsed empowerment values, even though achievement is maintained over training.

D.3 Pair-decoder parameterization.

We compare two logits:

$$\text{Difference encoder:} \quad f_\phi(s, s_+, z) = z^\top [\phi(s_+) - \phi(s)]/\tau,$$
$$\text{Pair decoder:} \quad f_\psi(s, s_+, z) = z^\top g_\psi([s, s_+])/\tau.$$

We use the same network architecture for both logit parameterizations. All other experiment details are the same. The pair decoder worsens training across most metrics *except* for the learned empowerment estimate, highlighting the importance of regularizing the classifier when doing empowerment maximization in scaled settings.

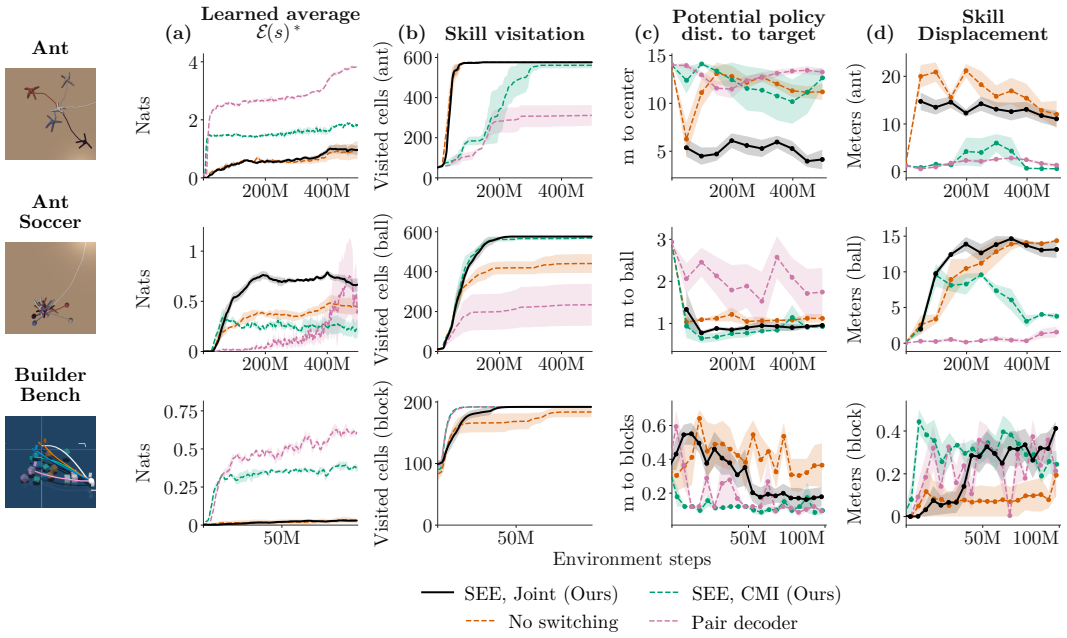


Figure 9: We compare the full method with variants that remove the two-phase rollout (no switching, orange), replace joint MI with conditional MI (green), or replace the bilinear logit with a general MLP (pair decoder, pink). The solid black curve shows SEE, and the colored dashed curves show the ablations. (a) For all methods, average empowerment increases over training. * Learned empowerment estimates use different objectives and regularization, so their magnitudes are not directly comparable across methods. (b) The skill visitation of SEE quickly saturates for these environments. Meanwhile, the ablations struggle to get visitation in a subset of the different environments. (c) SEE consistently learns potential policies that reach the center of the arena (Ant), approach the ball (AntSoccer), and reach the blocks (BuilderBench). While all variants successfully learn to reach the blocks in BuilderBench, the ablations all struggle to reach the center of the arena in Ant and some struggle to reach the ball in AntSoccer. (d) SEE consistently learns effective policies with large displacements. Five seeds for SEE; three per ablation.

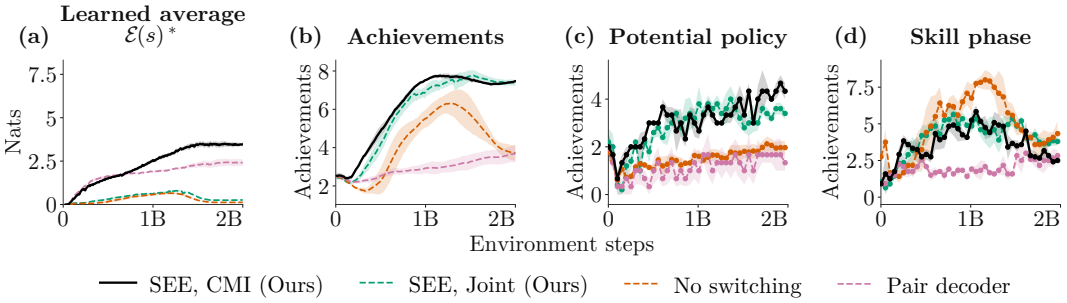


Figure 10: Craftax-Classic ablations (3 seeds; Joint MI: 5). The black curve is the SEE with the CMI classifier (Ours). The ablations are SEE with Joint MI (Ours), no switching (25% potential-policy episodes), and pair decoder. (a) Learned empowerment estimates rise over training for the base method. (b) Achievements per rollout. (c) New achievements in 100 potential-policy actions: 32 reset worlds per seed for no switching, and one saved prefix for the other methods. (d) New skill-phase achievements from four branches per seed/checkpoint; no-switching skills start from reset. Curves show mean $\pm$ standard error, with per-seed EMA ($\alpha = 0.05$) in (a,b) and unsmoothed checkpoints in (c,d).

E THEORY

Below, we include the key statement and proof that justifies the empowerment-maximization method. While the particular parameterizations of the underlying MISL methods to maximize potential empowerment is novel, we explicitly note that prior works such as VIC (Gregor et al., 2017) have already shown versions of the same theoretical results. We include these statements for completeness and emphasize that the primary contribution of our work is an empirical demonstration of tractable empowerment maximization, rather than novel theoretical results.

E.1 POLICY IMPROVEMENT OF THE POTENTIAL EMPOWERMENT LOWER-BOUND

The crux of (most) modern MISL methods is to learn a classifier that identifies skills from states, transitions, or trajectories, then use that classifier as a reward. In expectation, the cumulative reward estimates/lower bounds an MI (see Sections A.1 and 4.1). Policies that maximize this MI become interpretable skills. In our method, we use the resulting MI estimate as a reward, training a **potential empowerment policy** where agents navigate to states where they have many options. **We emphasize that this result is a modification of the result proved in Gregor et al. (2017). We do not claim that this is a significant theoretical contribution, and include these results for completeness to show that, under assumptions, our method gives policy improvement. In practical settings with function approximation, these assumptions do not hold.**

Here, we prove that alternating (1) fitting an on-policy skill classifier and (2) running policy improvement using the trained classifier as a reward improves a lower bound on the potential empowerment. The result requires two preliminaries. The first preliminary is the Barber-Agakov bound (Barber and Agakov, 2003) which is a variational bound on MI that comes from the positivity of the Kullback-Liebler Divergence; we have already invoked this bound when introducing the classifier rewards. The second preliminary is the statement that a standard MDP admits greedy policy improvement (Puterman, 2014).

We fix a start state s_0 and a skill prior $p(z)$ shared across iterations. The discounted future-state distribution is

$$p^{\pi}(s_+ \mid s_0, z) = (1-\gamma) \sum_{k=0}^{\infty} \gamma^k \Pr_{\pi}(S_{k+1} = s_+ \mid S_0 = s_0, Z = z).$$

For continuous states, the probabilities are replaced by densities.

Theorem 1 (Policy improvement on a lower bound of potential empowerment)**.** *Let $0 < \gamma < 1$ be the discount factor and let $\pi^{(n)}(a \mid s, z; s_0)$ be the current skill-conditioned policy at iteration n, with $z \sim p(z)$ sampled at s_0 and held fixed. Fit an on-policy classifier $q^{(n)}(z \mid s_0, s_+)$ to $p^{\pi^{(n)}}(z \mid s_0, s_+)$ and define the $(1-\gamma)$-normalized action value*

$$\begin{aligned} Q_z^{(n)}(s, a; s_0) &\triangleq Q_z^{\pi^{(n)}, q^{(n)}}(s, a; s_0) \\ &\triangleq (1-\gamma)\, \mathbb{E}_{\pi^{(n)}(a|s,z;s_0)} \left[\sum_{k=0}^{\infty} \gamma^k \log \frac{q^{(n)}(z \mid s_0, s_{t+k+1})}{p(z)} \middle| s_t = s,\ a_t = a \right]. \end{aligned}$$

Let $\pi^{(n+1)}$ be the result of a greedy policy-improvement step with respect to $Q_z^{(n)}(s, a; s_0)$ for every skill z and start-state s_0. If the classifier is perfectly fit on-policy at each iteration, $q^{(n)}(z \mid s_0, s_+) = p^{\pi^{(n)}}(z \mid s_0, s_+)$, then

$$I^{\pi^{(n+1)}}(Z; S_+ \mid S_0 = s_0) \geq I^{\pi^{(n)}}(Z; S_+ \mid S_0 = s_0) \qquad \forall n$$

which proves policy improvement under the assumptions of a perfectly-fit classifier.

Improving the *estimated potential empowerment-seeking behavior* reduces to standard RL with the same classifier intrinsic reward. Let $Q_z^{\pi,q}$ and $V_z^{\pi,q}$ denote values under policy π and classifier q

where n indexes training iterations, and $V_z^{\pi,q}(s;s_0) = \mathbb{E}_{a\sim\pi(\cdot|s,z;s_0)}[Q_z^{\pi,q}(s,a;s_0)]$. Then,

$$\widehat{\mathcal{E}_{\text{pot}}}^{(n)}(s) \approx \mathbb{E}_{p(z)}\left[V_z^{\pi^{(n)},q^{(n)}}(s;s)\right] \tag{11}$$

$$\leq I^{\pi^{(n)}}(Z;S_+ \mid S_0 = s) \tag{12}$$

$$\leq \mathcal{E}_{\text{pot}}(s) \tag{13}$$

$$= \max_{\pi\in\Pi_{\mathcal{Z}}} I^{\pi}(Z;S_+ \mid S_0 = s), \tag{14}$$

which, under the γ-discounted sum, is an unbiased Monte-Carlo estimate of a lower-bound on the cumulative potential empowerment objective:

$$\mathbb{E}_{\pi_{\text{pot}}}\left[\sum_{t\geq 0}\gamma^t \mathbb{E}_{z\sim p}\left[V_z^{\pi^{(n)},q^{(n)}}(s_t;s_t)\right]\right] \leq \mathbb{E}_{\pi_{\text{pot}}}\left[\sum_t \gamma^t \mathcal{E}_{\text{pot}}(s_t)\right].$$

Proposition 2 (A fixed classifier reward satisfies standard MDP conditions)**.** *Fix a classifier $q(z \mid s_0, s_+)$ and define the normalized intrinsic reward*

$$r_q(s_0,z,s') \triangleq (1-\gamma)\log\frac{q(z \mid s_0,s')}{p(z)}.$$

For any policy π, define

$$Q_z^{\pi,q}(s,a;s_0) \triangleq \mathbb{E}_{\pi(a|s,z;s_0)}\left[\sum_{k\geq 0}\gamma^k r_q(s_0,z,s_{t+k+1}) \,\middle|\, s_t = s, a_t = a\right].$$

Then $Q_z^{\pi,q}$ satisfies the Bellman equation

$$Q_z^{\pi,q}(s,a;s_0) = (1-\gamma)\,\mathbb{E}_{p(s'|s,a)}\left[\log\frac{q(z \mid s_0,s')}{p(z)}\right] + \gamma\,\mathbb{E}_{p(s'|s,a)}\left[V_z^{\pi,q}(s';s_0)\right],$$

where $V_z^{\pi,q}(s;s_0) = \mathbb{E}_{\pi(a|s,z;s_0)}Q_z^{\pi,q}(s,a;s_0)$.

Proof. Re-express the discounted sum in terms of the reward and remaining summation. The classifier is fixed, so r_q is a stationary reward for the MDP conditioned on skill and start-state (z, s_0). □

Proposition 3 (On-policy classifier gives tight lower bound on MI)**.** *For any policy π and any distibution $q(z \mid s_0, s_+)$,*

$$\mathcal{L}(\pi,q) \triangleq \mathbb{E}_{p(z)\,p_z^{\pi}(s_+|s_0)}\left[\log\frac{q(z \mid s_0,s_+)}{p(z)}\right] \leq I^{\pi}(Z;S_+ \mid S_0 = s_0).$$

Equality holds if $q(z \mid s_0, s_+) = p^{\pi}(z \mid s_0, s_+)$.

Proof. Follows from B-A bound.

□

Proof of Theorem 1. At iteration n, fit the classifier $q^{(n)}$ to the on-policy posterior:

$$\mathcal{L}(\pi^{(n)},q^{(n)}) = I^{\pi^{(n)}}(Z;S_+ \mid S_0 = s_0).$$

Now hold $q^{(n)}$ fixed. By Proposition 2, this is an ordinary MDP with stationary intrinsic reward. A greedy policy-improvement step therefore gives

$$\mathcal{L}(\pi^{(n+1)},q^{(n)}) \geq \mathcal{L}(\pi^{(n)},q^{(n)}).$$

Then, refit the classifier to the new on-policy posterior. By Proposition 3, the refitted classifier tightens the lower bound which can only increase the MI lower bound value:

$$\begin{aligned} I^{\pi^{(n+1)}}(Z; S_+ \mid S_0 = s_0) &= \mathcal{L}(\pi^{(n+1)}, q^{(n+1)}) \\ &\geq \mathcal{L}(\pi^{(n+1)}, q^{(n)}) \\ &\geq \mathcal{L}(\pi^{(n)}, q^{(n)}) \\ &= I^{\pi^{(n)}}(Z; S_+ \mid S_0 = s_0). \end{aligned}$$

Thus exact on-policy classifier fitting plus greedy improvement monotonically improves the true MI. □

Importantly, this theoretical result assumes an exact classifier and greedy policy updates at fixed s_0, which does not happen in practice with function approximation.

GLOSSARY

$\mathcal{E}_{\text{eff}}$ the effective empowerment as a function of the policy . 4, 5
$\mathcal{E}_{\text{pot}}$ the potential empowerment as a function of the state . 3–5, 24

InfoNCE information noise-contrastive estimation (Oord et al., 2018). 4

LatentGCE latent Gaussian-channel empowerment (Zhao et al., 2021). 1, 2, 6, 18

π_{eff} an effective empowerment maximizing policy . 4–7, 14
π_{pot} a potential empowerment maximizing policy . 3, 4, 6, 7, 14, 24

variational empowerment empowerment maximization with a variational bound and learned dynamics (Karl et al., 2017). 1
VIM variational information maximization (Mohamed and Jimenez Rezende, 2015). 1, 2, 6, 8, 18

ACRONYMS

CSF contrastive successor features. 4, 5, 12

DSOM discounted state occupancy measure. 3, 12

LQR linear-quadratic regulator. 19

MDP Markov decision process. 2, 3, 23, 24
METRA Metric-aware Abstraction. 4
MI mutual information. 2–8, 12, 13, 16, 21–25
MISL mutual information skill learning. 2–8, 12, 13, 23

RL reinforcement learning. 2, 23

USD unsupervised skill discovery. 2